Experiment No: 2

Name: Dipesh Makdiya

Class: D15C

Roll No: 32

Experiment:

Implement Multiple Linear Regression, Lasso, and Ridge Regression on real-world datasets

---

1. Dataset Source

Regression Dataset

Name: California Housing Dataset
Source: https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset
Access Method: `sklearn.datasets.fetch_california_housing`

This real-world dataset is used for implementing:

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression

---

2. Dataset Description

2.1 California Housing Dataset

Objective:
To predict the median house value in California districts.

Number of Instances: 20,640
Number of Features: 8

Features:

- MedInc – Median income in block group
- HouseAge – Median house age
- AveRooms – Average number of rooms
- AveBedrms – Average number of bedrooms
- Population – Block group population
- AveOccup – Average house occupancy
- Latitude – Latitude
- Longitude – Longitude

Target Variable:

MedHouseVal – Median house value (continuous)

Characteristics:

- Real-world dataset
- Numerical features
- No missing values
- Suitable for regression tasks

---

3. Mathematical Formulation of the Algorithms

3.1 Multiple Linear Regression

Multiple Linear Regression models the relationship between multiple input variables and a continuous output.

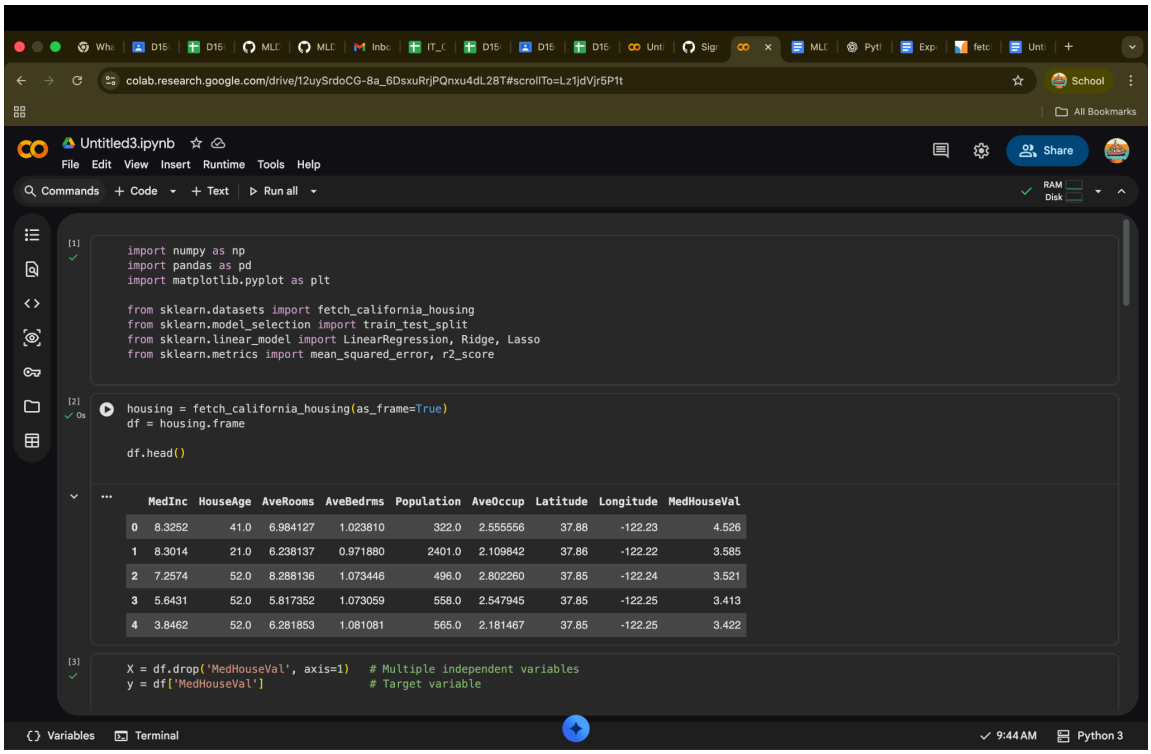
Hypothesis Function:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Cost Function (Mean Squared Error):

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The objective is to minimize the cost function to obtain optimal regression coefficients.

- 

3.2 Ridge Regression (L2 Regularization)

Ridge regression adds a penalty term to control overfitting.

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} \beta_j^2$$

Key Points:

- Reduces model variance
- Handles multicollinearity
- All coefficients are retained

-
```python
mlr = LinearRegression()
mlr.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```python
y_pred_mlr = mlr.predict(X_test)

print("Multiple Linear Regression")
print("MSE:", mean_squared_error(y_test, y_pred_mlr))
print("R2 Score:", r2_score(y_test, y_pred_mlr))
```

```
Multiple Linear Regression
MSE: 0.5558915986952444
R2 Score: 0.5757877060324508
```

```python
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
```

```
Ridge
Ridge()
```

3.3 Lasso Regression (L1 Regularization)

Lasso regression adds an absolute value penalty.

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} |\beta_j|$$

Key Points:

- Performs feature selection
- Some coefficients become zero
- Produces sparse models

colab.research.google.com/drive/12uySrdoCG-8a_6DsxuRrjPQnxu4dL28T#scrollTo=Lz1jdVjr5P1t

Untitled3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Lasso
Lasso(alpha=0.01)
```

[13]

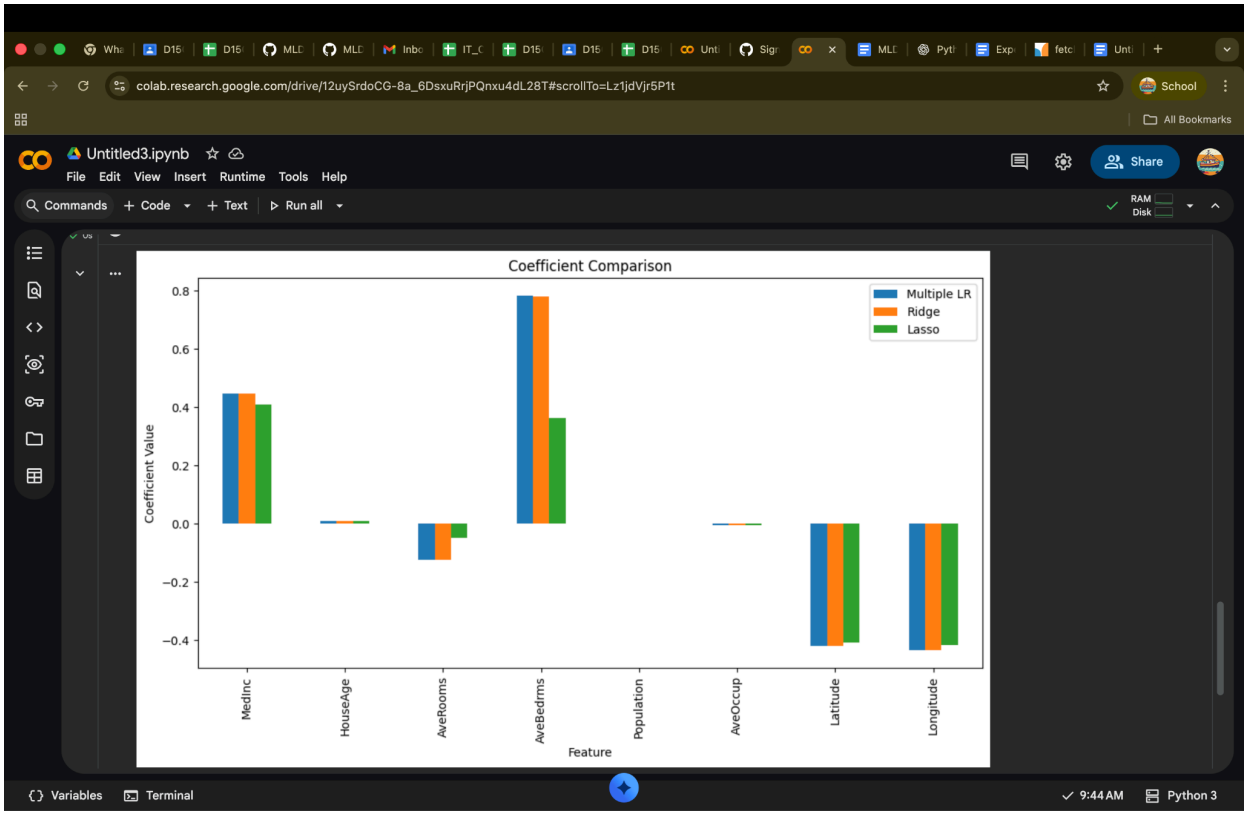
```
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Multiple LR': mlr.coef_,
    'Ridge': ridge.coef_,
    'Lasso': lasso.coef_
})

coefficients
```

	Feature	Multiple LR	Ridge	Lasso
0	MedInc	0.448675	0.448511	4.088956e-01
1	HouseAge	0.009724	0.009726	1.030849e-02
2	AveRooms	-0.123323	-0.123014	-4.744454e-02
3	AveBedrms	0.783145	0.781417	3.633460e-01
4	Population	-0.000002	-0.000002	-3.086013e-07
5	AveOccup	-0.003526	-0.003526	-3.359456e-03
6	Latitude	-0.419792	-0.419787	-4.071099e-01
7	Longitude	-0.433708	-0.433681	-4.149332e-01

Next steps: Generate code with coefficients New interactive sheet

Variables Terminal 9:44 AM Python 3



---

4. Algorithm Limitations

Multiple Linear Regression Limitations

- Assumes linear relationship between variables
- Sensitive to outliers
- Affected by multicollinearity
- Can overfit with large number of features

Ridge Regression Limitations

- Does not perform feature selection
- All variables remain in the model

Lasso Regression Limitations

- Can remove important features when λ is large

- Unstable when predictors are highly correlated

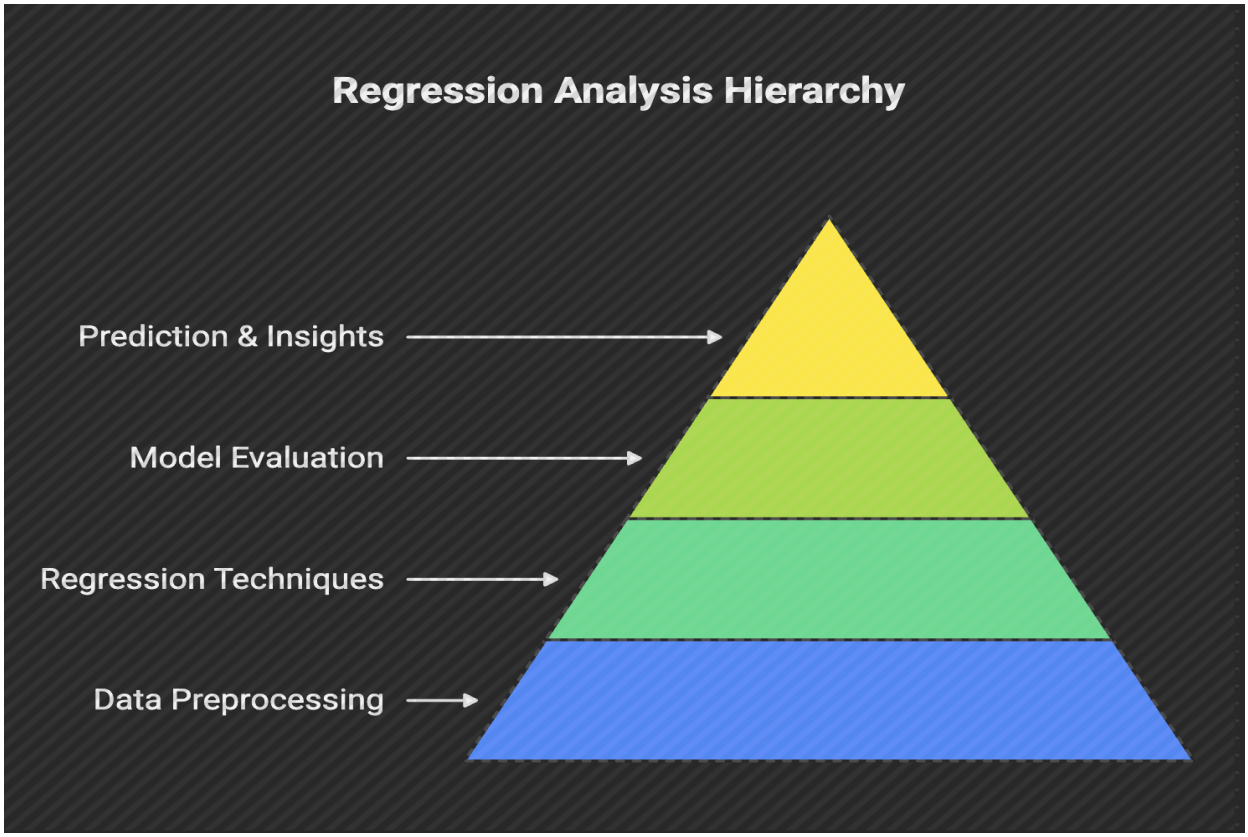
---

5. Methodology / Workflow

Step-by-Step Workflow:

1. Import required libraries
2. Load the California Housing dataset
3. Data exploration and preprocessing
4. Feature scaling using StandardScaler
5. Train-test split
6. Model training
    - Multiple Linear Regression
    - Ridge Regression
    - Lasso Regression
7. Prediction on test data
8. Performance evaluation
9. Hyperparameter tuning

Workflow Diagram:



Dataset → Preprocessing → Feature Scaling → Train/Test Split →
Model Training → Prediction → Evaluation

---

6. Performance Analysis

Evaluation Metrics Used:

Mean Squared Error (MSE)

- Lower value indicates better performance

R^2 Score

- Closer to 1 indicates better model fit

Interpretation:

- Multiple Linear Regression gives the baseline performance
- Ridge Regression reduces overfitting and improves generalization

- Lasso Regression performs feature selection and simplifies the model

---

7. Hyperparameter Tuning

Ridge Regression

Tuned Parameter:

- Alpha (λ)

Impact:

- Controls strength of regularization
- Higher value → More coefficient shrinkage → Less overfitting

---

Lasso Regression

Tuned Parameter:

- Alpha (λ)

Impact:

- Performs automatic feature selection
- Helps in reducing model complexity
- Improves test performance

---

8. Google Colab Link

EXPERIMENT 2.ipynb

---

9. Result

- Multiple Linear Regression successfully predicted the continuous house prices.
- Ridge Regression reduced overfitting and improved model stability.
- Lasso Regression selected the most important features by shrinking less important feature coefficients to zero.

---

10. Conclusion

This experiment demonstrates the implementation of Multiple Linear, Ridge, and Lasso Regression on a real-world dataset.
Regularization techniques help in improving model performance by reducing overfitting.
Lasso regression also performs automatic feature selection, making the model simpler and more interpretable.